Innovation Centre for Education

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A Constituent Unit of Yenepoya (Deemed to be University), Balmatta, Mangalore

Department of Computer Science

LOW-LEVEL DESIGN (LLD)

on

"Identity Security Framework for Retail"

Student Name: Liyakath Sulfeekar

Register Number: 23BCCE110

Programme: BCA – Cyber Security, Ethical Hacking & Cloud Computing with IBM

Industry Mentor: Shashank Guided by: Munisah

Table of Contents

1. Introduction..... 3
1.1. Scope..... 3
1.2. Intended Audience..... 3
2. System Design..... 3
2.1. Application Design..... 3
2.2. Process Flow..... 4
2.3. ComponentsDesign..... 5
2.4. Key Design Desicions..... 5
2.5. API Catalogue..... 6
3. Data Design.....7
3.1. Database Tables..... 7
3.2. Data Access..... 9
3.3. Data Retention..... 9
4. Interfaces..... 9
5. Session Management..... 9
6. Non-Functional Requirements..... 10
6.1 Security……..……10
6.2 Performance……...10
7. References.....11

1. INTRODUCTION

This LLD describes how the Identity Security Framework for Retail was actually built. It covers the database tables, route functions, validation rules, and security mechanisms. The structure follows the same IBM format as the HLD, with Caching and Data Migration omitted since they do not apply to this project.

1.1 Scope

This document covers the file structure, all ten database tables, each route and what it does, form validation rules, error handling, and the password and OTP mechanisms.

1.2 Intended Audience

Written for the internal guide Munisah, academic evaluators, and the IBM industry mentor.

2. SYSTEM DESIGN

2.1 Application Design

The application runs from a single file — app.py. Flask handles all routing. SQLite stores the data. Jinja2 renders the HTML pages. There are no external frameworks or libraries apart from Werkzeug for password hashing and smtplib for sending OTP emails.

Layer	What it does
Flask (app.py)	Handles HTTP requests, checks sessions, calls the database, returns HTML pages
SQLite (database.db)	Stores all data — users, logs, orders, inventory, complaints, salary, attendance
Jinja2 Templates	Renders every page the user sees in the browser
Werkzeug	Hashes passwords with PBKDF2-SHA256 and verifies them during login
Gmail SMTP	Sends OTP verification emails to users

2.2 Process Flow

- Signup: user fills form → password is hashed → OTP sent to email → user enters OTP → account saved to database
- Login: password checked → OTP sent → user enters OTP → session created → redirected to role dashboard
- Password reset: email entered → OTP sent → OTP verified → new password saved
- Account lockout: 5 wrong passwords → account locked 10 minutes → auto-unlocked after time expires

2.3 Components

- Authentication — signup, login with 2FA, forgot/reset password, account lockout
- Admin — view all users, unlock/delete accounts, view login logs, post announcements
- Staff — inventory, reports, working hours, salary, complaints, announcements
- Customer — my orders, invoice, profile, reward score, announcements, complaint box

2.4 Key Design Decisions

- Single file (app.py) — keeps the project simple and easy to run on any machine
- SQLite — no server needed, database is just one file created automatically on first run
- Manual session handling — written from scratch to show understanding of how authentication works
- Email OTP — chosen over authenticator apps because every user already has an email
- Generic login errors — same message whether username or password is wrong, to prevent user enumeration
- POST-Redirect-GET — used on all forms to prevent double submission on browser refresh

2.5 API Catalogue

No external APIs are used. All communication is standard HTTP form submissions. The only external service is Gmail SMTP for OTP emails. The full list of internal routes is below.

URL	Method	Who can access / What it does
/login	GET, POST	Public — login form and password check
/signup	GET, POST	Public — registration with OTP email
/verify_signup	GET, POST	Session — OTP check for new account
/verify_login_otp	GET, POST	Session — second factor OTP check
/forgot / /otp / /reset	GET, POST	Public — forgot password flow
/logout	GET	Anyone logged in — clears session
/dashboard	GET	Customer — home with stats and links
/my_orders	GET, POST	Customer — place orders, view history
/invoice/<id>	GET	Customer — printable order invoice
/profile	GET, POST	Customer — view and update email
/change_password	GET, POST	All roles — change password

/my_announcements / /complaints	GET, POST	Customer — announcements and complaints
/staff	GET	Staff — home dashboard
/inventory	GET, POST	Staff — manage stock, add/delete products
/reports	GET	Staff — daily, weekly, monthly sales
/work_hours	GET, POST	Staff — clock in and clock out
/salary	GET	Staff — monthly salary records
/staff_complaints	GET	Staff — view and resolve complaints
/announcements	GET	Staff — view announcements
/admin	GET	Admin — users, logs, stats
/unlock/<user> / /delete/<user>	GET	Admin — unlock or delete accounts
/announcements/add	POST	Admin — post new announcement

3. DATA DESIGN

3.1 Database Tables

All ten tables are created automatically by init_db() when the app starts for the first time.

Table	What it stores
users	All accounts — username, hashed password, email, role (Admin/Staff/Customer), failed login count, lock timestamp
login_logs	Every login attempt — who, which IP, what time, success or failure
inventory	Products — name, category, quantity in stock, price, low stock alert level
sales	Sales recorded by staff — product, quantity sold, total amount, time
returns	Product returns logged by staff — product, quantity, reason, status
customer_orders	Orders placed by customers — product, quantity, total price, timestamp
complaints	Customer complaints — subject, message, open or resolved status
announcements	Admin notices visible to staff and customers
staff_hours	Attendance — clock-in time, clock-out time, hours worked per day
salary	Monthly salary records for each staff member — amount, paid status, date

Key columns in the users table: failed_attempts (integer, increments on each wrong password), lock_time (set when account is locked, NULL when active), is_verified (set to 1 after email OTP is confirmed at signup), password (always a PBKDF2-SHA256 hash — never plain text).

3.2 Data Access

All database queries use Python's sqlite3 module with ? parameterised placeholders to prevent SQL injection. Each route opens a connection, does its work, and closes it before sending the response. Columns are accessed by name using the sqlite3.Row row factory.

3.3 Data Retention

No automated deletion is set up since this is a local academic project. All records stay until the database file is deleted. In a real deployment, login logs would be cleared after 90 days and salary records kept for 7 years.

4. INTERFACES

The only interface is the web browser. All pages are plain HTML and CSS rendered server-side. There are no JavaScript frameworks. The only external connection is to Gmail SMTP (port 587, STARTTLS) for sending OTP emails. No mobile app, no REST API, no webhooks.

5. SESSION MANAGEMENT

Flask stores the session in a signed browser cookie. During multi-step flows (signup, login 2FA, password reset), the OTP and form data sit in the session and are deleted once the step is done. After a successful login the session holds the username and role until logout.

Errors are passed between pages using two helper functions: set_error() saves a message to the session before a redirect, and get_error() reads it and deletes it on the next page load. This means each error shows exactly once and disappears if the user refreshes.

6. NON-FUNCTIONAL REQUIREMENTS

6.1 Security

- Passwords stored as PBKDF2-SHA256 hashes — plain passwords are never written to the database
- Email OTP required on every login as a second authentication factor
- Account locks after 5 wrong password attempts for 10 minutes
- Each protected route checks session role before doing anything
- OTPs expire after 5 minutes
- Reusing the same password in reset or change password is blocked
- Every login attempt is logged with IP address and timestamp

6.2 Performance

The application runs locally for demonstration. Pages load quickly because everything is server-side with no JavaScript frameworks. OTP email delivery takes 1 to 3 seconds due to SMTP. All user lookups use UNIQUE indexed columns so database queries are fast.

7. REFERENCES

- Flask Documentation — https://flask.palletsprojects.com
- Werkzeug Security — https://werkzeug.palletsprojects.com/en/latest/utils/
- OWASP Authentication Cheat Sheet — https://cheatsheetseries.owasp.org
- Python sqlite3 Docs — https://docs.python.org/3/library/sqlite3.html
- NIST SP 800-63B — https://pages.nist.gov/800-63-3/sp800-63b.html